RSU-Assisted Dual-Channel Imputation for IoV Trajectory Prediction in Mixed HDV/CAV Traffic

Problem Statement

Background and Context

The emergence of Connected and Autonomous Vehicles (CAVs) is transforming highway traffic management. CAVs communicate with each other and with roadside infrastructure using Vehicle-to-Everything (V2X) communication technology, broadcasting their speed, position, and acceleration ten times per second (10 Hz). This real-time data sharing enables sophisticated cooperative services, the most critical of which is **trajectory prediction**: a system that uses the last 3 seconds of observed vehicle motion (30 time steps) to forecast where each vehicle will be in the next 5 seconds (50 time steps). Accurate trajectory prediction is the foundation of cooperative lane-changing, collision avoidance, and platoon management in modern Intelligent Transportation Systems (ITS).

However, there is a fundamental problem the research community has not addressed: **CAVs do not operate in a world of only other CAVs.**

The Real-World Problem

During the current and near-future deployment period (expected to last 10-20 years), CAVs share the road with a majority of Human-Driven Vehicles (HDVs). HDVs have no V2X communication capability. They do not broadcast their speed or position. They are, from a V2X perspective, completely invisible. At current and near-future market penetration rates of 10-30% CAV, a typical CAV on the highway is surrounded predominantly by HDVs that generate no V2X signal whatsoever.

This creates a critical data quality crisis for trajectory prediction. The trajectory predictor's input window (the 30-step observation matrix) must contain not just the ego-CAV's own state (speed, acceleration, position), but also the **surrounding vehicles' state**: the gap to the vehicle ahead (leader), the leader's speed, and the follower's speed. These relative-state features allow the predictor to model interaction dynamics, the most important factor in highway trajectory evolution.

When surrounding vehicles are HDVs, **these features are permanently absent**. There is no historical signal. There is no partial observation. The entire feature column is structurally missing for the full 30-step window. This is called **Structural Feature Missingness (SFM)** and it cannot be solved by any standard imputation method (linear interpolation, KNN, GRU-D), because all of these require at least some past observation of the missing feature. When the feature has never been observed, classical methods have nothing to work from.

Simultaneously, even the ego-CAV's own state data suffers from a second type of missingness: **Temporal Cluster Missingness (TCM)**. V2X communication is not perfectly reliable. Channel interference, RSU handover, and burst packet loss create short bursts of missing observations (typically 2-5 consecutive steps) in the ego-vehicle's own state history. Unlike SFM, TCM is temporal (some observations exist around the gap), but the burst pattern is fundamentally different from the random missingness assumed by most imputation literature.

Neither SFM nor TCM fits the standard missing data taxonomy (MCAR, MAR, MNAR). They are **IoV-native missing patterns** generated by the communication architecture of the vehicular network. No existing imputation framework is designed for them.

What Existing Literature Covers (and Where It Falls Short)

The imputation research community has produced excellent methods for fixed-sensor traffic data (loop detectors, PeMS stations): tensor completion, GNNs, diffusion models. These assume a fixed sensor either has a reading or does not. They do not apply to mobile vehicles with dynamic neighborhoods.

In the trajectory prediction community, TrajImpute (NeurIPS 2024) and MS-TIP (ICML 2024) have begun combining imputation with trajectory prediction, but only for pedestrians, with randomly missing data, and without any IoV communication architecture. WRN (IJCAI 2024) addresses autonomous vehicle trajectory prediction with missing observations but uses random masking with no V2X context and no infrastructure-side data source.

No existing paper addresses: SFM from HDV non-participation in V2X, TCM from V2X burst packet loss, RSU sensing to fill structurally absent neighbor features, or imputation quality evaluated through downstream trajectory prediction accuracy.

The Proposed Solution

This work proposes an **RSU-assisted dual-channel imputation framework** that reconstructs the corrupted 30-step trajectory prediction input window before it reaches the predictor.

The key insight is that while HDVs do not broadcast V2X messages, they are **physically visible** to Roadside Unit (RSU) infrastructure sensors (cameras and radar already deployed at roadside). The RSU detects and tracks HDVs, estimates their speed and position, and transmits this information to nearby CAVs via V2I communication. The RSU thus acts as a **virtual neighbor** supplying the surrounding-vehicle state that V2V communication cannot provide.

Channel 1 (SFM Handler): RSU observes surrounding HDVs through its camera, estimates their kinematic state using a Kalman Filter, and delivers gap and speed estimates to the CAV via V2I. These fill the structurally absent neighbor-state feature columns.

Channel 2 (TCM Handler): A physics-constrained GRU-D model reconstructs burst-missing segments in the ego-CAV's own state history using kinematic priors (speed bounds, acceleration limits).

Both channels produce a complete 30x7 imputed input matrix for the trajectory predictor. Quality is evaluated on reconstruction accuracy (MAE, RMSE) AND downstream prediction performance (ADE, FDE), directly linking imputation quality to vehicular service quality.

Why This Is Not Just Data Preprocessing

1. **SFM cannot be solved by any interpolation method** - there are zero observed values in the feature column to interpolate from. A different data source (RSU camera) is structurally required. This is a sensing architecture problem, not a data cleaning problem.
2. **The missing mechanism is domain-specific** - SFM is caused by fleet composition (HDV/CAV ratio) and TCM by V2X channel physics. General-purpose imputation methods do not model these causes.
3. **Validation is end-to-end** - imputation quality is measured through downstream trajectory prediction accuracy, making this a system-level vehicular intelligence contribution.

Implementation Steps - Overview

1. Build the SUMO highway simulation environment
2. Extract ground truth trajectory windows via TraCI
3. Run simulations across three CAV penetration rates
4. Inject Structural Feature Missingness (SFM) into windows
5. Inject Temporal Cluster Missingness (TCM) into windows
6. Simulate RSU camera observations of HDVs
7. Implement Channel 1 - SFM Handler (Kalman Filter)
8. Implement Channel 2 - TCM Handler (Physics-constrained GRU-D)
9. Integrate both channels into the dual-channel imputer
10. Implement and train the LSTM trajectory predictor
11. Run the full experiment matrix
12. Conduct HighD supplementary validation
13. Analyse and document results

Detailed Step-by-Step Implementation

Step 1: Build the SUMO Highway Simulation Environment

What to do: Create a SUMO simulation of a straight 3-lane highway segment, 1 km long, with mixed CAV and HDV traffic and a virtual RSU at the midpoint.

Why this step is needed: SUMO gives complete, noise-free ground truth data for ALL vehicles including HDVs because SUMO simulates all of them internally. This is the critical advantage

over real datasets: we know the true HDV state at every step, which we later mask to simulate SFM and then compare against for evaluation. Real datasets like NGSIM do not provide clean HDV V2X data because HDVs never had V2X capability in the first place.

Details:

Network file `highway.net.xml`: 3 lanes, each 3.5m wide, 1000m length, speed limit 33.33 m/s (120 km/h).

Traffic demand file `mixed_traffic.rou.xml`: two vehicle types (CAV with CACC car-following model, HDV with Krauss model), flow of ~1500 vehicles/hour/lane, penetration rate as a parameter.

```
<vType id="CAV" carFollowModel="CACC" minGap="1.0" accel="3.0" decel="5.0"/>
<vType id="HDV" carFollowModel="Krauss" minGap="2.5" accel="2.6" decel="4.5"/>
```

RSU coverage zone: 300m radius around the 500m midpoint. Only vehicles within this zone are included.

Run SUMO at 10 Hz:

```
sumo --step-length 0.1 --net-file highway.net.xml --route-files mixed_traffic.rou.xml
```

Step 2: Extract Ground Truth Trajectory Windows via TraCI

What to do: Use the TraCI Python API to extract, at every simulation step, the state of every CAV in the RSU zone and its surrounding vehicles. Organise data into 30-step sliding windows with 50-step future trajectory labels.

Why this step is needed: The 30-step input window and 50-step prediction label define the exact input-output structure of the trajectory predictor. Extracting in this format from the start ensures everything downstream operates on consistent data structures. The ground truth windows are the reference against which all imputed windows are compared.

Details:

```
import traci
import numpy as np

WINDOW = 30
RSU_CENTER = 500.0
RSU_RADIUS = 300.0

history_buffer = {}
dataset = []
type_metadata = []

traci.start(['sumo', '-c', 'highway.sumocfg'])
```

```
for step in range(6000):  # 600 seconds at 10 Hz
    traci.simulationStep()
    for veh_id in traci.vehicle.getIDList():
        x = traci.vehicle.getPosition(veh_id)[0]
        if abs(x - RSU_CENTER) > RSU_RADIUS:
            continue
        leader   = traci.vehicle.getLeader(veh_id)
        follower = traci.vehicle.getFollower(veh_id)
        snap = {
            'ego_speed':      traci.vehicle.getSpeed(veh_id),
            'ego_accel':      traci.vehicle.getAcceleration(veh_id),
            'ego_x':          traci.vehicle.getPosition(veh_id)[0],
            'ego_y':          traci.vehicle.getPosition(veh_id)[1],
            'gap_to_leader':  leader[1] if leader else np.nan,
            'leader_speed':   traci.vehicle.getSpeed(leader[0]) if leader else np.nan
            'follower_speed': traci.vehicle.getSpeed(follower[0]) if follower else np
            'leader_type':    traci.vehicle.getTypeID(leader[0]) if leader else 'none
            'follower_type':  traci.vehicle.getTypeID(follower[0]) if follower else
        }
        if veh_id not in history_buffer:
            history_buffer[veh_id] = []
        history_buffer[veh_id].append(snap)
        if len(history_buffer[veh_id]) >= WINDOW:
            buf = history_buffer[veh_id][-WINDOW:]
            window = np.array([[s['ego_speed'], s['ego_accel'],
                                s['ego_x'], s['ego_y'],
                                s['gap_to_leader'], s['leader_speed'],
                                s['follower_speed']] for s in buf])
            dataset.append(window)
            type_metadata.append({
                'leader':   [s['leader_type']   for s in buf],
                'follower': [s['follower_type']  for s in buf]
            })

traci.close()
np.save('ground_truth.npy', np.array(dataset))
```

Each window: shape (30, 7). Features: [ego_speed, ego_accel, ego_x, ego_y, gap_to_leader, leader_speed, follower_speed]. Save type_metadata separately for SFM injection in Step 4.

Step 3: Run Simulations Across Three CAV Penetration Rates

What to do: Repeat Step 2 with penetration rates of 10%, 20%, and 30% CAV. Run 20 simulations per configuration with different random seeds.

Why this step is needed: Penetration rate is the primary experimental variable. At 10% CAV, approximately 90% of surrounding vehicles are HDVs, meaning SFM affects nearly all neighbor features. Running 20 seeds per configuration ensures results are statistically robust and not dependent on one specific random traffic realisation.

Details: Create three route files with different CAV/HDV vehicle ratios. Automate runs:

```
for penetration in [10, 20, 30]:
    for seed in range(20):
        run_simulation(
            route_file=f'traffic_{penetration}pct.rou.xml',
            seed=seed,
            output=f'data/gt_pct{penetration}_seed{seed}.npy'
        )
```

Expected output: 60 dataset files (3 penetrations x 20 seeds), each containing thousands of clean 30x7 windows with vehicle type metadata.

Step 4: Inject Structural Feature Missingness (SFM)

What to do: Post-process the ground truth datasets. For every time step where a surrounding vehicle is an HDV, mask the corresponding neighbor-state features (columns 4, 5, 6) to NaN.

Why this step is needed: In reality, a CAV cannot receive any data from an HDV via V2X because HDVs have no V2X radio. We simulate this by reading the vehicle type metadata from Step 2 and nullifying the features that would be absent in real deployment. Ground truth values remain stored separately for evaluation. This gives perfect experimental control: we know exactly what the correct answer is and can compute imputation error precisely.

Details:

```
def inject_sfm(window, vehicle_types):
    corrupted = window.copy().astype(float)
    sfm_mask  = np.ones((30, 7), dtype=float)
    for t in range(30):
        if vehicle_types['leader'][t] in ['HDV', 'none']:
            corrupted[t, 4] = np.nan
            corrupted[t, 5] = np.nan
            sfm_mask[t, 4]  = 0
            sfm_mask[t, 5]  = 0
        if vehicle_types['follower'][t] in ['HDV', 'none']:
            corrupted[t, 6] = np.nan
            sfm_mask[t, 6]  = 0
    return corrupted, sfm_mask
```

Expected missing rate by penetration: 10% CAV -> ~90% NaN in neighbor columns, 20% CAV -> ~80% NaN, 30% CAV -> ~70% NaN. This is precisely why linear interpolation fails: there are no observed endpoints in the column to interpolate between.

Step 5: Inject Temporal Cluster Missingness (TCM)

What to do: Apply burst-pattern missing data to the ego-vehicle's own state features (columns 0-3: speed, acceleration, x, y) using a Gilbert-Elliott two-state Markov model that generates realistic V2X packet loss bursts.

Why this step is needed: V2X packet loss occurs in bursts, not as isolated random drops. When the wireless channel enters a bad state due to interference or handover, several consecutive packets fail before recovery. The Gilbert-Elliott model captures this physics with two parameters: probability of entering a bad state and probability of exiting it. A physically realistic missing pattern ensures the experiment reflects true V2X conditions and the imputer trains on the correct distribution of gaps.

Details:

```
def inject_tcm(series, p_enter_bad=0.05, p_exit_bad=0.4):
    # Mean burst length = 1 / p_exit_bad = 2.5 steps = 250ms
    corrupted = series.copy().astype(float)
    tcm_mask  = np.ones(30, dtype=float)
    state = 0  # 0=GOOD, 1=BAD
    for t in range(30):
        if state == 0 and np.random.random() < p_enter_bad:
            state = 1
        elif state == 1 and np.random.random() < p_exit_bad:
            state = 0
        if state == 1:
            corrupted[t] = np.nan
            tcm_mask[t]  = 0
    return corrupted, tcm_mask

def inject_tcm_to_window(window, p_enter=0.05, p_exit=0.4):
    corrupted = window.copy()
    tcm_mask  = np.ones((30, 7), dtype=float)
    for col in range(4):  # ego-state columns only
        corrupted[:, col], tcm_mask[:, col] = inject_tcm(
            window[:, col], p_enter, p_exit)
    return corrupted, tcm_mask
```

Typical TCM missing rate per ego-feature column: approximately 11%, clustered in bursts of 2-5 steps.

Step 6: Simulate RSU Camera Observations of HDVs

What to do: For each HDV surrounding a CAV, generate a simulated RSU camera observation by taking the SUMO ground truth HDV position and speed and degrading it with Gaussian measurement noise and occasional missed detections.

Why this step is needed: Channel 1 uses RSU sensor data as input. Since we are working in simulation, we approximate real RSU camera output by adding physically calibrated noise to SUMO ground truth HDV state. Noise parameters (position error ~0.5m, speed error ~0.3 m/s, detection rate ~95%) are taken from published roadside camera benchmarks. This gives a realistic input signal for Channel 1 without requiring real RSU hardware.

Details:

```
def simulate_rsu_camera(ground_truth_hdv_state,
                        position_noise_std=0.5,
                        speed_noise_std=0.3,
                        detection_rate=0.95):
    rsu_obs = ground_truth_hdv_state.copy().astype(float)
    for t in range(30):
        if np.random.random() > detection_rate:
            rsu_obs[t, :] = np.nan  # missed detection
        else:
            rsu_obs[t, 0] += np.random.normal(0, position_noise_std)
            rsu_obs[t, 1] += np.random.normal(0, speed_noise_std)
    return rsu_obs
```

Store rsu_observations alongside corrupted_window, sfm_mask, and tcm_mask in dataset files. Each sample now has five components: ground_truth, corrupted_input, sfm_mask, tcm_mask, rsu_obs.

Step 7: Implement Channel 1 - SFM Handler (Kalman Filter)

What to do: Implement a Kalman Filter that takes noisy RSU camera observations of an HDV and produces a smoothed estimate of the HDV position and speed. Use this estimate to compute gap_to_leader and leader_speed, filling the SFM-masked feature columns.

Why this step is needed: RSU camera observations contain measurement noise and occasional gaps from missed detections. Feeding raw noisy values to the predictor would degrade performance. The Kalman Filter combines a physical constant-velocity motion model with noisy measurements to produce a minimum-variance state estimate. It is computationally trivial (microseconds per step), suitable for real-time 10 Hz operation, and automatically propagates estimates through missed-detection gaps without any additional logic.

Details:

```
from filterpy.kalman import KalmanFilter

def build_hdv_kf():
    kf = KalmanFilter(dim_x=4, dim_z=2)  # state=[x,vx,y,vy], obs=[x,y]
    dt = 0.1
    kf.F = np.array([[1,dt, 0, 0],[0, 1, 0, 0],[0, 0,1,dt],[0, 0,0, 1]])
    kf.H = np.array([[1,0,0,0],[0,0,1,0]])
    kf.R = np.eye(2) * 0.25   # camera noise
    kf.Q = np.eye(4) * 0.1
    kf.P *= 10
    return kf

def sfm_impute_channel1(rsu_obs, ego_positions):
    kf = build_hdv_kf()
    first = next((t for t in range(30) if not np.isnan(rsu_obs[t,0])), None)
    if first is None:
        return np.full(30, np.nan), np.full(30, np.nan)
    kf.x = np.array([rsu_obs[first,0], 0.0, rsu_obs[first,1], 0.0])
    gaps, speeds = np.zeros(30), np.zeros(30)
```

```
    for t in range(30):
        kf.predict()
        if not np.isnan(rsu_obs[t,0]):
            kf.update(rsu_obs[t,:])
        gaps[t]   = kf.x[0] - ego_positions[t, 0]
        speeds[t] = kf.x[1]
    return gaps, speeds
```

After Channel 1, columns 4 and 5 of the input window are filled with physically consistent Kalman-smoothed estimates derived from RSU camera observations.

Step 8: Implement Channel 2 - TCM Handler (Physics-Constrained GRU-D)

What to do: Implement a GRU-D neural network that reconstructs burst-missing segments in the ego-vehicle's own state history. The model uses time-since-last-observation as an additional input signal and clamps its output to physically valid vehicle kinematic bounds.

Why this step is needed: GRU-D was designed specifically for time series with missing values. Its decay mechanism is physically appropriate for vehicles: when observations are missing for longer, the hidden state decays toward a learned baseline rather than rigidly holding the last observed value. Kinematic constraints (max speed 40 m/s, acceleration -5 to +3 m/s^2) ensure that no physically impossible values are output, which would corrupt the predictor input.

Details:

```
import torch, torch.nn as nn, torch.nn.functional as F

class GRU_D_Cell(nn.Module):
    def __init__(self, input_size, hidden_size):
        super().__init__()
        self.gru_cell = nn.GRUCell(input_size * 2, hidden_size)
        self.gamma    = nn.Linear(input_size, input_size)
        self.baseline = nn.Parameter(torch.zeros(input_size))
    def forward(self, x, mask, delta_t, h):
        decay = torch.exp(-F.relu(self.gamma(delta_t)))
        x_hat = decay * h[:, :x.size(1)] + (1 - decay) * self.baseline
        x_in  = mask * x + (1 - mask) * x_hat
        h_new = self.gru_cell(torch.cat([x_in, mask], dim=-1), h)
        return x_in, h_new

class TCMImputer(nn.Module):
    def __init__(self, feature_dim=4, hidden_dim=64):
        super().__init__()
        self.cell   = GRU_D_Cell(feature_dim, hidden_dim)
        self.fc     = nn.Linear(hidden_dim, feature_dim)
        self.register_buffer('lo', torch.tensor([0.0, -5.0, -1e6, -1e6]))
        self.register_buffer('hi', torch.tensor([40.0, 3.0,  1e6,  1e6]))
    def forward(self, x, mask):
        h, out = torch.zeros(x.size(0), 64).to(x.device), []
        dt = self._delta_t(mask)
        for t in range(30):
            xt = x[:, t, :].nan_to_num(0)
```

```
            xi, h = self.cell(xt, mask[:,t,:], dt[:,t,:], h)
            o = torch.clamp(self.fc(h), self.lo, self.hi)
            out.append(mask[:,t,:] * xt + (1 - mask[:,t,:]) * o)
        return torch.stack(out, dim=1)
    def _delta_t(self, mask):
        B, T, F = mask.shape
        dt = torch.zeros_like(mask)
        for t in range(1, T):
            dt[:,t,:] = torch.where(mask[:,t-1,:]==1,
                                    torch.zeros(B,F,device=mask.device),
                                    dt[:,t-1,:]+1)
        return dt
```

Training: MSE loss computed only at masked (missing) positions. Adam optimiser, lr=1e-3, 100 epochs, batch size 64, 80/20 train/val split.

Step 9: Integrate Both Channels into the Dual-Channel Imputer

What to do: Combine Channel 1 and Channel 2 into a single callable module. Channel 1 runs first (fills structural HDV gaps from RSU), then Channel 2 (fills ego-state bursts). Output is a complete 30x7 matrix with no NaN values.

Why this step is needed: The two channels address different missing types in different feature groups and must run in correct order. Packaging them as a single module simplifies all experiment code, makes the contribution clearly presentable in the paper, and enables clean ablation testing in Step 11.

Details:

```
class DualChannelImputer:
    def __init__(self, tcm_model_path):
        self.tcm = TCMImputer()
        self.tcm.load_state_dict(torch.load(tcm_model_path))
        self.tcm.eval()

    def impute(self, corrupted, sfm_mask, tcm_mask, rsu_obs, ego_pos):
        w = corrupted.copy().astype(float)
        # Channel 1: SFM - fill neighbor features from RSU Kalman
        g, ls = sfm_impute_channel1(rsu_obs, ego_pos)
        w[sfm_mask[:,4]==0, 4] = g[sfm_mask[:,4]==0]
        w[sfm_mask[:,5]==0, 5] = ls[sfm_mask[:,5]==0]
        w[sfm_mask[:,6]==0, 6] = np.nanmean(w[:,0])  # follower: use mean speed
        # Channel 2: TCM - fill ego-state bursts with GRU-D
        xt = torch.tensor(np.nan_to_num(w[:,:4],nan=0.0),dtype=torch.float32).unsque
        mt = torch.tensor(tcm_mask[:,:4],dtype=torch.float32).unsqueeze(0)
        with torch.no_grad():
            w[:,:4] = self.tcm(xt, mt).squeeze(0).numpy()
        return w  # complete 30x7, no NaN
```

Step 10: Implement and Train the LSTM Trajectory Predictor

What to do: Implement a standard two-layer LSTM encoder that takes a complete 30-step window and predicts the ego-vehicle x,y positions for the next 50 time steps. Train once on complete ground truth windows. Freeze weights permanently.

Why this step is needed: The predictor is the measuring instrument, not the research contribution. By training once on complete data and freezing, all differences in prediction error across conditions (no imputation / imputed / oracle) are caused purely by input window quality. Using a standard LSTM ensures reproducibility without specialised resources.

Details:

```
class LSTMPredictor(nn.Module):
    def __init__(self, input_dim=7, hidden=128, pred_steps=50):
        super().__init__()
        self.enc = nn.LSTM(input_dim, hidden, num_layers=2, batch_first=True, dropou
        self.fc  = nn.Linear(hidden, pred_steps * 2)
        self.ps  = pred_steps
    def forward(self, x):
        _, (h, _) = self.enc(x)
        return self.fc(h[-1]).view(-1, self.ps, 2)

# Train once on complete ground truth, then freeze
predictor = LSTMPredictor()
for epoch in range(200):
    for x_complete, y_future in train_loader:
        loss = F.mse_loss(predictor(x_complete), y_future)
        loss.backward(); optimiser.step(); optimiser.zero_grad()
for p in predictor.parameters(): p.requires_grad = False
predictor.eval()
```

Three evaluation conditions per experiment:

- **Condition A:** corrupted window (NaN -> 0), no imputation
- **Condition B:** dual-channel imputed window
- **Condition C:** complete ground truth (oracle upper bound)

Metrics: ADE (Average Displacement Error) and FDE (Final Displacement Error) in meters.

Step 11: Run the Full Experiment Matrix

What to do: Execute five systematic experiments answering the five research questions. For each, compare conditions A, B, C and five baselines.

Why this step is needed: Each experiment answers a specific research question. Together they prove: the problem is real and quantifiable (E1); it is caused by structural missingness not random noise (E3); standard methods cannot solve it (E4); both channels are necessary (E5); results hold across penetration rates (E2). Without this structure, the paper has results but no argument.

Details:

E1 - Missing Rate Sensitivity: Fix 20% penetration, vary missing rate 10%-90%. Plot ADE vs. missing rate for A, B, C. Expected: A rises sharply; B stays near C.

E2 - Penetration Rate Sensitivity: Fix 50% missing, vary penetration 10%, 20%, 30%. Expected: gap between A and B largest at 10% (most SFM).

E3 - Missing Pattern Decomposition: SFM only / TCM only / SFM+TCM. Expected: SFM causes larger ADE than TCM; baselines fail completely on SFM.

E4 - Method Comparison:

Baseline	Type	Handles SFM?
Zero-fill	Trivial	No
Forward-fill	Classical	No
Linear interpolation	Classical	No
GRU-D (temporal only)	Deep Learning	No
SAITS	Attention SOTA	No
Proposed dual-channel	RSU + KF + GRU-D	**Yes**

Key claim: all baselines fail on SFM because they have no external data source for missing columns.

E5 - Ablation Study: Channel 1 only / Channel 2 only / Full dual-channel. Confirms individual and joint contribution of each channel.

Run each experiment 5 times with different seeds. Report mean +/- standard deviation. Use paired t-test ($p < 0.05$) to confirm statistical significance vs. best baseline.

Step 12: Conduct HighD Supplementary Validation

What to do: Apply the trained dual-channel imputer without retraining to windows extracted from the HighD real-world vehicle trajectory dataset. Measure imputation MAE and prediction ADE/FDE.

Why this step is needed: A simulation-only study raises the legitimate reviewer question: does this work on real vehicle trajectories? HighD provides real high-precision vehicle positions on German highways, giving a real-data sanity check without requiring changes to the method. Using HighD as supplementary (not primary) is appropriate because we still simulate RSU observations using the same noise model from Step 6, since no real RSU data exists in HighD.

Details:

1. Download HighD from highd-dataset.com (free for academic use)
2. Downsample from 25 Hz to 10 Hz

3. Extract 30-step windows with 50-step labels (same window logic as Step 2)
4. Randomly designate 30% of vehicles as CAV, 70% as HDV by vehicle ID
5. Apply SFM injection (Step 4) based on labels
6. Apply TCM injection (Step 5) with same parameters
7. Simulate RSU observations using Step 6 noise model
8. Run DualChannelImputer.impute() with no parameter changes
9. Run frozen LSTM predictor, compute ADE/FDE
10. Report in a separate table with one paragraph of discussion

Step 13: Analyse and Document Results

What to do: Compile all results into tables and figures, compute statistical significance, and write the paper following the structure below.

Why this step is needed: Results without clear presentation do not constitute a contribution. Tables and figures are how reviewers evaluate claims. Statistical testing is mandatory at Q1 venues.

Key tables: (1) E4 method comparison: MAE, RMSE, ADE, FDE for all methods. (2) E5 ablation: channel contribution. (3) E2 penetration sensitivity. (4) HighD supplementary.

Key figures: (1) System architecture diagram. (2) ADE vs. missing rate curve (E1). (3) Sample window visualisation (corrupted vs. imputed vs. ground truth). (4) E4 method comparison bar chart. (5) E5 ablation bar chart.

Paper structure:

1. Introduction
2. Related Work (imputation, V2X trajectory prediction, cooperative perception)
3. Problem Formulation (SFM and TCM definitions, input window structure)
4. Proposed Method (dual-channel architecture, Channel 1 KF, Channel 2 GRU-D)
5. Experiments (SUMO setup, missing injection, E1-E5)
6. Supplementary Validation (HighD)
7. Conclusion

Implementation Timeline

Phase	Activity	Duration
Phase 1	SUMO scenario + TraCI extraction (Steps 1-3)	Weeks 1-4
Phase 2	Missing data injection - SFM, TCM, RSU sim (Steps 4-6)	Weeks 5-6
Phase 3	Channel 1: Kalman Filter (Step 7)	Weeks 7-8

Phase	Activity	Duration
Phase 4	Channel 2: GRU-D model - implement, train, debug (Step 8)	Weeks 9-13
Phase 5	Dual-channel integration (Step 9)	Week 14
Phase 6	LSTM predictor - train and freeze (Step 10)	Weeks 15-16
Phase 7	Full experiment matrix E1-E5 (Step 11)	Weeks 17-21
Phase 8	HighD supplementary (Step 12)	Week 22
Phase 9	Analysis, figures, paper writing (Step 13)	Weeks 23-26

Total: 26 weeks

Software and Tools Required

Tool	Purpose	Installation
SUMO (v1.18+)	Traffic simulation	sumo.dlr.de
TraCI (Python)	SUMO programmatic control	pip install traci
NumPy / Pandas	Data processing	pip install numpy pandas
filterpy	Kalman Filter	pip install filterpy
PyTorch	GRU-D and LSTM	pytorch.org
scikit-learn	Baseline models and metrics	pip install scikit-learn
matplotlib	Figures and plots	pip install matplotlib
SciPy	Statistical testing	pip install scipy